

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 5

«Формирование изображения методом трассировки путей»

Выполнил
студент

Стрельбицкий Илья Павлович

Группа № Р3413

Преподаватель: Жданов Дмитрий Дмитриевич

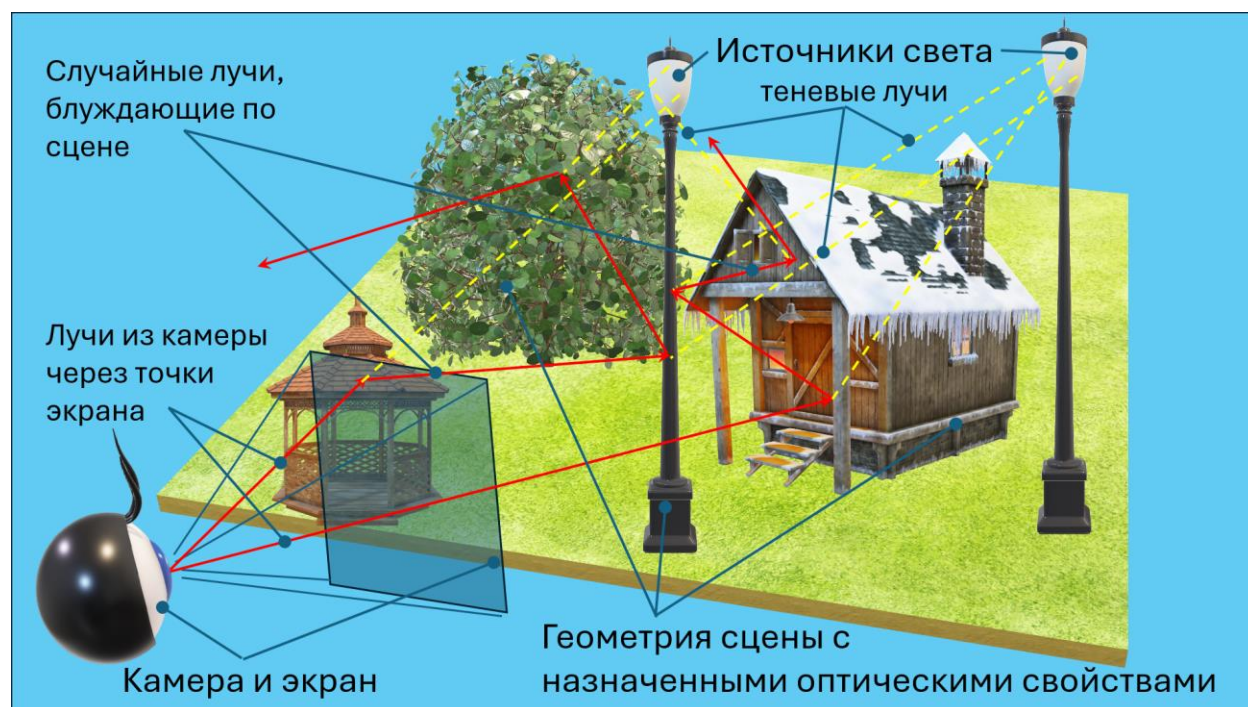
г. Санкт-Петербург 2026

Цель

Освоить простейшие методы синтеза изображений трехмерных сцен с учетом глобального освещения методом трассировки путей. Убедиться и доказать корректность сформированного изображения.

Задание

Построить изображение трехмерной сцены с корректным учетом глобального освещения методом трассировки путей.



Исходные данные и допущения

- Модель геометрии сцены, формируется треугольной сеткой. Треугольная сетка может быть сформирована вручную или импортирована из OBJ файла. В любом случае необходимо понимать как модель будет выглядеть на изображении. Для поиска точки встречи луча с поверхностью можно написать свою функцию, можно воспользоваться библиотекой Embree (или любой другой библиотекой).
- Цветовая модель – RGB.
- Модель оптических свойств материалов. Пропускание отсутствует, среда равномерная и прозрачная с постоянным показателем преломления $n = 1$. Оптические свойства на поверхностях: коэффициент диффузного отражения (Ламберт) и коэффициент зеркального отражения (чистое

зеркало). Зеркальные и диффузные оптические свойства могут быть заданы одновременно на одной поверхности. Выбирается одно событие (свойство) методами выборки по значимости и русской рулетки. Коэффициенты цветные. Новое направление и цвет луча выбираются в соответствии с выбранным событием. Желательно обеспечить постоянство энергии луча. Более сложные модели (например, Фонг) возможны, но не обязательны. Обязательное условие – физичность модели, т.е. суммарное отражение для каждой компоненты цвета не превышает 1.

- Камера точечная, смотрит и видит сцену. Разрешение до 1000x1000 точек, но не менее 500x500. Антиалиасинг обеспечивается случайным выбором координат точки в пределах одного пикселя.
- Источники света – протяженные, диаграмма излучения – Ламберт. Источники цветные. Расчет прямого освещения можно ограничить одним случайным лучом со случайно выбранного источника света. Использовать выборку по значимости в соответствии с мощностью источника света, току на источнике выбирать случайным образом, Вариация яркости по поверхности треугольника отсутствует.
- Продолжительность расчета можно задать временем, точностью или числом лучей, выпущенных с каждого пикселя изображения.
- Изображение формируется в абсолютных яркостях и потом переводится в область относительных яркостей с нормировкой на некоторое значение (максимальную яркость, среднюю яркость на уровне 0.5, заранее заданную или какую-либо еще). Все, что больше 1, отсекается до уровня 1. Далее применяется гамма-коррекция $V_{corr} = V^{1/\gamma}$ (обычно $\gamma = 2.2$), результат приводится к уровню 0-255 и записывается в файл изображения. Простейший формат PPM.

Результат представит в виде отчета, с описанием алгоритмов и методов, используемых в программе, текста программы (с пониманием текста), и файлов изображений. Программа должна иметь достаточно простой интерфейс (любого вида) для возможности изменить параметры сцены и получить новый результат в процессе защиты работы.

Ход работы

1. Введение и постановка задачи

Трассировка путей (Path Tracing) – это метод Монте-Карло для решения уравнения рендеринга (Rendering Equation), впервые предложенный Джеймсом Каджией в 1986 году. Задача лабораторной работы заключалась в реализации алгоритма трассировки путей для создания фотореалистичных изображений трехмерных сцен с корректным учетом глобального освещения.

2. Теоретические основы

Основой физически корректного рендеринга является уравнение рендеринга Каджи:

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) \cos\theta_i d\omega_i$$

Где:

- $L_o(x, \omega_o)$ – исходящая излучаемость (radiance) из точки x в направлении ω_o
- $L_e(x, \omega_o)$ – собственная излучаемость поверхности (emission)
- $f_r(x, \omega_i, \omega_o)$ – BRDF (Bidirectional Reflectance Distribution Function)
- $L_i(x, \omega_i)$ – входящая излучаемость из направления ω_i
- $\cos\theta_i = n \cdot \omega_i$ – косинус угла между нормалью и направлением падения
- Ω – полусфера всех возможных направлений

Уравнение рендеринга решается методом Монте-Карло через оценку интеграла:

$$L_o \approx \left(\frac{1}{N}\right) \sum_{j=1}^N [f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) \cos\theta_i] / p(\omega_i)$$

Где $p(\omega_i)$ – плотность вероятности выборки направления ω_i .

3. Реализованная модель BRDF

В данной работе реализована гибридная модель BRDF с вероятностным выбором компонент (Probability Mixture BRDF).

BRDF представляется как смесь двух компонент:

$$f_r(\omega_i \rightarrow \omega_o) = \begin{cases} \frac{k_d}{\pi} * \frac{n * \omega_i}{p_d} \text{ с вероятностью } p_d = \frac{L_d}{L_d + L_s} \\ k_s * \frac{\delta(\omega_r - \omega_i)}{p_s} \text{ с вероятностью } p_s = \frac{L_s}{L_d + L_s} \end{cases}$$

Где:

- k_d – коэффициент диффузного отражения (Lambert)
- k_s – коэффициент зеркального отражения
- δ – дельта функция Дирака (для идеального зеркала)
- L_d, L_s – яркости диффузной и зеркальной компонент

Класс Material обеспечивает физическую корректность через проверку:

```
def is_physical(self) -> bool:
    """Проверка, что сумма коэффициентов отражения не превышает 1."""
    total = self.total_reflectance()
    return total.r <= 1.0 and total.g <= 1.0 and total.b <= 1.0
```

Это гарантирует выполнение **закона сохранения энергии**: $k_d + k_s \leq 1$

Физическая корректность материала проверяется при его создании, и если материал некорректный, то программа автоматически масштабирует коэффициенты.

4. Алгоритмы выборки (Sampling Techniques)

4.1. Важность материалов (Material Importance Sampling)

Для выбора между диффузным и зеркальным отражением используется вероятностный подход:

```
diffuse_prob = diffuse_weight / total_weight if total_weight > 0 else 0.5
specular_prob = specular_weight / total_weight if total_weight > 0 else 0.5
```

Вероятность пропорциональна вкладу каждой компоненты в общую яркость.

4.2. Косинусная выборка для диффузных поверхностей

Для диффузных поверхностей (модель Ламберта) используется косинусная выборка: $pdf(\omega) = \frac{\cos\theta}{\pi}$

В коде это реализовано в методе `Vector3.random_cosine_direction(normal)` через преобразование: $r_1, r_2 \in [0,1]$; $\varphi = 2\pi r_1$; $\sin\theta = \sqrt{r_2}$; $\cos\theta = \sqrt{1 - r_2}$

Это дает равномерное распределение по полусфере с весом, пропорциональным косинусу угла.

4.3. Выборка прямого освещения

Для снижения шума в текущем коде используется метод Next Event Estimation (NEE). Вклад от прямого освещения вычисляется независимо для каждого типа отражения и учитывается внутри вероятностной выборки BRDF.

Формула для диффузного отражения:

$$L_{direct} = \frac{L_{light} * f_{r_{diffuse}} * \cos\theta * \cos\theta_{light}}{r^2 * p_{light} * p_{diffuse}}$$

Где:

- L_{light} – излучаемость источника света
- $f_{r_{diffuse}} = \frac{k_d}{\pi}$ – диффузная компонента BRDF
- $\cos\theta = |n * \omega_{light}|$ – косинус между нормалью поверхности и направлением к свету (учет ориентации)
- $\cos\theta_{light} = |n_{light} * (-\omega_{light})|$ – косинус между нормалью источника и направлением к точке
- r – расстояние до источника света
- $p_{light} = \frac{1}{N_{lights} * area_{light}}$ – вероятность выборки данного источника
- $p_{diffuse}$ – вероятность выбора диффузного отражения

Для зеркального отражения прямое освещение не учитывается, поскольку таковы математические свойства модели идеального зеркала.

Идеальное зеркало описывается BRDF, содержащей дельта-функцию Дирака:

$$f_{r_{specular}}(\omega_i \rightarrow \omega_o) = k_s * \delta(\omega_r - \omega_i)$$

Где $\omega_r = reflect(\omega_o, n)$ – идеально отраженное направление.

Интеграл уравнения рендеринга для зеркала:

$$L_{specular} = \int_{\Omega} k_s * \delta(\omega_r - \omega_i) * L_i(\omega_i) * \cos\theta_i d\omega_i$$

Свойство дельта-функции «выбирает» только одно направление:

$$L_{specular} = k_s * L_i(\omega_r) * \cos\theta_r$$

NEE предполагает явную выборку источников света. Однако для идеального зеркала:

- Направление ω_r детерминировано (полностью определяется падающим направлением ω_o и нормалью)
- Вероятность того, что случайно выбранный источник света точно совпадет с направлением ω_r , равна нулю
- Следовательно, оценка прямого освещения через выборку источников не работает

Для идеального зеркала учитываются только два типа путей:

- Прямая видимость источника: если источник находится точно в направлении отражения
- Отраженная видимость: источник виден через отражение в других поверхностях

4.4. Русская рулетка

Для ограничения длины путей используется вероятностное прекращение трассировки:

```
if depth > 3: # Начинаем применять после 3-го отскока
    survival_prob = 0.8
    if random.random() > survival_prob:
        return Spectrum.black()
else:
    survival_prob = 1.0
```

Этот метод сохраняет математическое ожидание:

$$E[L] = p * \frac{L}{p} + (1 - p) * 0 = L$$

Сохранение математического ожидания: чтобы компенсировать преждевременное обрывание путей и не вносить смещение (bias), вклад оставшихся путей усиливается в $1/p$ раз. Это гарантирует, что в среднем энергия сохраняется.

```
if depth > 3:
    result = result / survival_prob
```

5. Геометрические вычисления и оптимизации

5.1. Ускорение пересечений с использованием trimesh

Вместо наивного перебора $O(N)$ используется библиотека trimesh, которая реализует BVH (Bounding Volume Hierarchy) со сложностью $O(\log N)$.

Кэширование структуры сцены:

```
def _build_trimesh_scene(self):  
    if self.trimesh_cache is not None:  
        return self.trimesh_cache  
    self.trimesh_cache = { . . . }
```

Идея BVH:

Bounding Volume Hierarchy (BVH) – древовидная структура данных, которая рекурсивно разделяет пространство сцены на ограничивающие объемы (AABB – Axis-Aligned Bounding Box).

Основная идея:

- Группируем объекты в ограничивающие объемы
- Если луч не пересекает объем, пропускаем все объекты внутри
- Рекурсивно применяем к дочерним узлам

5.2. Вычисление барицентрических координат

Барицентрические координаты – это система координат для однозначного задания положения точки внутри треугольника относительно его вершин. Это ключевой инструмент для плавной интерполяции атрибутов (нормали, цвет, текстурные координаты) по поверхности треугольника.

- **Суть:** Любую точку P внутри треугольника с вершинами A, B, C можно представить как взвешенную сумму:

$$P = u \cdot A + v \cdot B + w \cdot C$$

где u, v, w – барицентрические координаты, причём $u + v + w = 1$ и $u, v, w \geq 0$.

- **Интерпретация:** Координата u показывает «близость» точки к вершине A , v – к B , w – к C . В вершине A : ($u=1, v=0, w=0$).

В нашем коде это используется для интерполяции нормалей:

1. При пересечении луча с треугольником находим точку пересечения P и её барицентрические координаты (u, v, w).
2. У треугольника есть нормали в вершинах: n_A, n_B, n_C
3. Нормаль в точке P вычисляется как:

$$normal_at_P = (u * n_A + v * n_B + w * n_C).normalize()$$

Зачем это нужно? Если использовать одну и ту же (например, усреднённую) нормаль для всего треугольника, то сфера, аппроксимированная треугольниками, будет выглядеть как гранёный многогранник. Плавная интерполяция нормалей между вершинами создаёт иллюзию идеально гладкой криволинейной поверхности, что критически важно для визуального качества.

5.3. Интерполяция нормалей

Нормаль в точке пересечения вычисляется как взвешенная сумма вершинных нормалей:

```
light_normal = (  
    light.normals[0] * w +  
    light.normals[1] * u +  
    light.normals[2] * v  
) .normalize()
```

6. Модель камеры и антиалиасинг

6.1. Перспективная камера

Камера реализует перспективную проекцию с параметрами:

- Положение (position)
- Направление (target)
- Вектор вверх (up)
- Поле зрения (fov_degrees)

```
u = (pixel_x / self.width - 0.5) * 2.0  
v = (0.5 - pixel_y / self.height) * 2.0  
point_on_screen = (  
    self.screen_center +  
    self.screen_u * u +  
    self.screen_v * v  
)  
direction = (point_on_screen - self.position).normalize()
```

6.2. Антиалиасинг

Для устранения ступенчатых артефактов используется случайное смещение внутри пикселя:

```
offset_x = random.random() - 0.5  
offset_y = random.random() - 0.5
```

7. Обработка изображения

7.1. Тонмаппинг

Тонмаппинг (Tone Mapping): это процесс преобразования (сжатия) широкого диапазона яркостей HDR-изображения в узкий диапазон LDR-устройства, с максимальным сохранением визуального восприятия деталей и контраста.

Физически корректный рендеринг производит HDR-изображения. Тонмаппинг нужен чтобы преобразовать их в LDR.

Функция `tonemap` в текущем коде реализует приближение оператора ACES (Academy Color Encoding System) – отраслевого стандарта для преобразования HDR в LDR, обеспечивающего естественное и визуально приятное сжатие динамического диапазона:

$$L_{display} = \frac{L_{scene} * exposure * (A * L_{scene} * exposure + B)}{L_{scene} * exposure * (C * L_{scene} * exposure + D) + E}$$

Где $A=2.51$, $B=0.03$, $C=2.43$, $D=0.59$, $E=0.14$ – эмпирические константы, подобранные для оптимального визуального восприятия.

7.2. Гамма-коррекция

Для корректного отображения на дисплеях применяется гамма-коррекция:

$$L_{sRGB} = L_{linear}^{\frac{1}{\gamma}}, \text{ где } \gamma \approx 2.2 \text{ для стандарта sRGB.}$$

8. Структура программы

Реализованная программа в рамках лабораторной представляет собой самостоятельную реализацию на языке Python с использованием библиотек NumPy для математических операций и trimesh для ускорения геометрических вычислений.

8.1. Структура классов:

- Vector3 - математические операции с 3D векторами
- Spectrum - представление цвета и операций с ним
- Ray - луч с началом и направлением
- Triangle - треугольный примитив с интерполяцией нормалей
- Material - оптические свойства поверхности
- Camera - перспективная камера
- Scene - управление геометрией с ускорением через trimesh
- PathTracer - основной алгоритм трассировки путей
- CornellBoxScene - фабрика для создания тестовой сцены

8.2. Алгоритм трассировки пути

Основной алгоритм в методе `trace_path()`:

1. Базовые условия: если глубина $\geq \text{max_depth}$, вернуть черный цвет (`Spectrum.black()`).
2. Поиск пересечения: используется ускоренный метод `scene.intersect(ray)` с `trimesh`. Если пересечений нет, вернуть черный цвет.
3. Проверка на источник света: если пересечение произошло с излучающей поверхностью (`hit.is_light_source`), немедленно вернуть ее излучательность (`hit.emission`).
4. Russian Roulette: При глубине трассировки больше 3 ($\text{depth} > 3$) путь с вероятностью 20% обрывается. Вероятность выживания `survival_prob = 0.8`.
5. Вычисление вероятностей для выбора BRDF:
 - a. `diffuse_weight = material.diffuse.luminance`
 - b. `specular_weight = material.specular.luminance`
 - c. `diffuse_prob = diffuse_weight / (diffuse_weight + specular_weight)`
 - d. `specular_prob = 1 - diffuse_prob`
6. Вероятностный выбор и расчет:
 - a. Если выбрано диффузное отражение (с вероятностью `diffuse_prob`):
 - i. Косвенное освещение: генерируется новое направление `new_direction` по косинусному распределению (`Vector3.random_cosine_direction`). Рекурсивно вычисляется вклад `incoming`. Вес Монте-Карло: $(\text{brdf} * \cosine) / (\text{pdf} * \text{diffuse_prob})$, где $\text{pdf} = |\text{new_direction} \cdot \text{normal}| / \pi$.

- ii. Прямое освещение (NEE): вычисляется одним сэмплом в `sample_direct_lighting`. Его вклад умножается на `brdf` и делится на `diffuse_prob`.
 - iii. Итог: `result = indirect_light + direct_light`.
 - b. Если выбрано зеркальное отражение (с вероятностью `specular_prob`):
 - i. Косвенное освещение: Направление детерминировано – `reflected = ray.direction.reflect(normal)`. Вес Монте-Карло: `material.specular / specular_prob`.
 - ii. Прямое освещение для зеркальных путей не учитывается, так как для идеального зеркала вероятность попадания в точечный источник света со случайного направления равна нулю.
7. Компенсация Russian Roulette: если путь выжил на шаге 4, его окончательный вклад делится на `survival_prob`, сохраняя математическое ожидание.

8.3. Параметры сцены Cornell Box:

- Материалы:
 - Левая стена: красный диффузный
 - Правая стена: зеленый диффузный
 - Задняя стена: белый диффузный, слегка зеркальный
 - Пол, потолок: белый диффузный
 - Левая сфера: белый диффузный материал
 - Правая сфера: зеркальный материал
- Источник света: белый прямоугольный источник на потолке (2 «светящихся» треугольника)

8.4. Интерфейс программы:

Программа реализована как консольное приложение с поддержкой аргументов командной строки для гибкой настройки параметров рендеринга:

- Разрешение изображения (`--width, --height`)
- Количество сэмплов на пиксель (`--samples`)
- Максимальная глубина трассировки (`--max-depth`)
- Поле зрения камеры (`--fov`)
- Параметры тонмэппинга (экспозиция, гамма) (`--exposure, --gamma`)

- Имя выходного файла (--output)

```
D:\university\spi\ifth\venv\Scripts\python.exe D:\university\spi\ifth\main.py --width 600 --height 600 --samples 24 --max-depth 8
Разрешение:      600x600
Сэмплов на пиксель: 24
Глубина трассировки: 8
Поле зрения камеры: 45.0
Экспозиция:      1.0
Гамма-коррекция: 2.2

Создание сцены...
Материал 'Red Material' физически корректен
Материал 'Green Material' физически корректен
Материал 'White Material' физически корректен
Материал 'Mirror White Material' физически корректен
Материал 'Mirror Material' физически корректен
Материал 'Light' физически корректен
Треугольников:   172
Источников света: 2

Начало рендеринга...
Начало рендеринга: 600x600
Сэмплов на пиксель: 24
Глубина трассировки: 8
Треугольников в сцене: 172
```

Пример запуска программы

8.5. Пример использования:

```
python path_tracer.py --width 500 --height 500 --samples 16 --max-depth 8 --exposure 1.5
```

8.6. Процесс рендеринга:

Программа выводит прогресс рендеринга в реальном времени с указанием процента выполнения и затраченного времени. Рендеринг может занимать значительное время в зависимости от выбранных параметров (от нескольких минут для тестовых сцен до часов для высококачественных изображений).

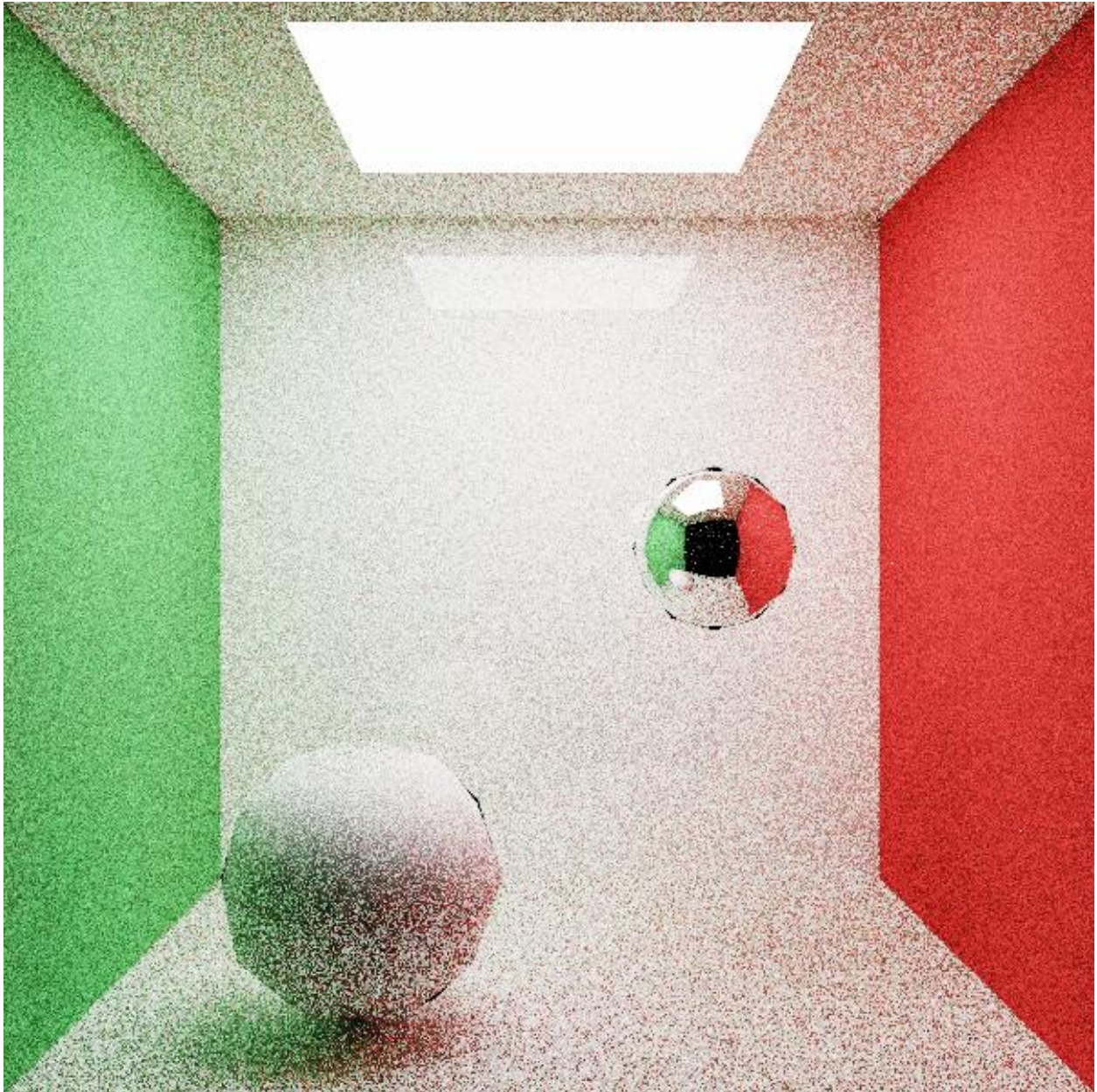
8.7. Результат:

На выходе программа формирует PPM-файл с готовым изображением, которое демонстрирует корректное глобальное освещение, включая:

- Мягкие тени от протяженных источников света
- Цветное взаимное отражение (color bleeding)
- Зеркальные отражения
- Естественное затухание света в глубине сцены

Для наглядного примера были заданы следующие параметры:

```
python path_tracer.py --width 600 --height 600 --samples 24 --max-depth 8
```



Сгенерированное изображение

Исходный код программы на Python:

```
import numpy as np
import trimesh

import math
import random
import time
import sys
import argparse
from datetime import datetime
from typing import Optional, Tuple

# =====
# Базовые математические структуры
# =====
class Vector3:
    """Трёхмерный вектор с основными операциями."""
    def __init__(self, x: float = 0.0, y: float = 0.0, z: float = 0.0):
        self.x = float(x)
        self.y = float(y)
        self.z = float(z)

    def __repr__(self) -> str:
        return f"Vector3({self.x:.4f}, {self.y:.4f}, {self.z:.4f})"

    def __add__(self, other: 'Vector3') -> 'Vector3':
        return Vector3(self.x + other.x, self.y + other.y, self.z + other.z)

    def __sub__(self, other: 'Vector3') -> 'Vector3':
        return Vector3(self.x - other.x, self.y - other.y, self.z - other.z)

    def __mul__(self, scalar: float) -> 'Vector3':
        return Vector3(self.x * scalar, self.y * scalar, self.z * scalar)

    def __rmul__(self, scalar: float) -> 'Vector3':
        return self * scalar

    def __truediv__(self, scalar: float) -> 'Vector3':
        return Vector3(self.x / scalar, self.y / scalar, self.z / scalar)

    def __neg__(self) -> 'Vector3':
        return Vector3(-self.x, -self.y, -self.z)

    def dot(self, other: 'Vector3') -> float:
        return self.x * other.x + self.y * other.y + self.z * other.z

    def cross(self, other: 'Vector3') -> 'Vector3':
        return Vector3(
            self.y * other.z - self.z * other.y,
            self.z * other.x - self.x * other.z,
            self.x * other.y - self.y * other.x
        )

    @property
    def length(self) -> float:
```

```

        return math.sqrt(self.dot(self))

    def normalize(self) -> 'Vector3':
        norm = self.length
        if norm > 0:
            return self / norm
        return self

    def reflect(self, normal: 'Vector3') -> 'Vector3':
        return self - normal * (2 * self.dot(normal))

    def __getitem__(self, idx: int) -> float:
        if idx == 0:
            return self.x
        if idx == 1:
            return self.y
        if idx == 2:
            return self.z
        raise IndexError(f"Index {idx} out of range")

    @staticmethod
    def random_cosine_direction(normal: 'Vector3') -> 'Vector3':
        """Генерация случайного вектора по косинусному распределению."""
        r1 = random.random()
        r2 = random.random()
        phi = 2 * math.pi * r1
        sin_theta = math.sqrt(r2)
        cos_theta = math.sqrt(1 - r2)
        x = math.cos(phi) * sin_theta
        y = math.sin(phi) * sin_theta
        z = cos_theta
        if abs(normal.x) > abs(normal.y):
            tangent = Vector3(normal.z, 0, -normal.x).normalize()
        else:
            tangent = Vector3(0, -normal.z, normal.y).normalize()
        bitangent = normal.cross(tangent)
        return (tangent * x + bitangent * y + normal * z).normalize()

# =====
# Объекты свойств цвета и света
# =====

class Spectrum:
    """Спектральный цвет с RGB-компонентами."""
    def __init__(self, r: float = 0.0, g: float = 0.0, b: float = 0.0):
        self.r = float(r)
        self.g = float(g)
        self.b = float(b)

    def __repr__(self) -> str:
        return f"Spectrum({self.r:.4f}, {self.g:.4f}, {self.b:.4f})"

    def __add__(self, other: 'Spectrum') -> 'Spectrum':
        return Spectrum(self.r + other.r, self.g + other.g, self.b + other.b)

    def __sub__(self, other: 'Spectrum') -> 'Spectrum':
        return Spectrum(self.r - other.r, self.g - other.g, self.b - other.b)

```



```

    def __mul__(self, other) -> 'Spectrum':
        if isinstance(other, Spectrum):
            return Spectrum(self.r * other.r, self.g * other.g, self.b *
other.b)
        return Spectrum(self.r * other, self.g * other, self.b * other)

    def __rmul__(self, scalar: float) -> 'Spectrum':
        return self * scalar

    def __truediv__(self, scalar: float) -> 'Spectrum':
        return Spectrum(self.r / scalar, self.g / scalar, self.b / scalar)

    @property
    def luminance(self) -> float:
        """Вычисление яркости цвета."""
        return 0.2126 * self.r + 0.7152 * self.g + 0.0722 * self.b

    @staticmethod
    def black() -> 'Spectrum':
        """Черный цвет."""
        return Spectrum(0, 0, 0)

    @staticmethod
    def white() -> 'Spectrum':
        """Белый цвет."""
        return Spectrum(1, 1, 1)

class Ray:
    """Луч с начальной точкой и направлением."""
    def __init__(self, origin: Vector3, direction: Vector3):
        self.origin = origin
        self.direction = direction.normalize()

class Material:
    """Оптические свойства материала поверхности"""
    def __init__(
        self,
        diffuse: Spectrum,
        specular: Spectrum,
        emission: Spectrum = None,
        name: str = ""
    ):
        self.diffuse = diffuse
        self.specular = specular
        self.emission = emission if emission else Spectrum.black()
        self.name = name
        # проверка физической корректности материала и исправление если нужно
        self._validate_materials()

    def is_emissive(self) -> bool:
        """Проверка, является ли материал источником света"""
        return self.emission.r > 0 or self.emission.g > 0 or
self.emission.b > 0

    def total_reflectance(self) -> Spectrum:
        """Суммарная отражательная способность"""
        return Spectrum(

```

```

        self.diffuse.r + self.specular.r,
        self.diffuse.g + self.specular.g,
        self.diffuse.b + self.specular.b
    )

def is_physical(self) -> bool:
    """Проверка, что сумма коэффициентов отражения не превышает 1"""
    total = self.total_reflectance()
    return total.r <= 1.0 and total.g <= 1.0 and total.b <= 1.0

def _validate_materials(self) -> None:
    """Проверка, что все материалы физически корректны"""
    if self.is_physical():
        print(f"Материал '{self.name}' физически корректен")
        return
    print(
        f"Материал '{self.name}' НЕ физически корректен! Сумма
коэффициентов: {self.total_reflectance()}"
    )
    self._make_physical()
    print(f"Исправлено: новые коэффициенты diff={self.diffuse},
spec={self.specular}")

def _make_physical(self, max_reflectance: float = 0.99) -> None:
    """Исправление на физически корректную версию материала"""
    total = self.total_reflectance()
    # Масштабируем каждый канал независимо
    scale_r = max_reflectance / total.r if total.r > 1.0 else 1.0
    scale_g = max_reflectance / total.g if total.g > 1.0 else 1.0
    scale_b = max_reflectance / total.b if total.b > 1.0 else 1.0
    # Применяем масштабирование
    new_diffuse = Spectrum(
        self.diffuse.r * scale_r,
        self.diffuse.g * scale_g,
        self.diffuse.b * scale_b
    )
    new_specular = Spectrum(
        self.specular.r * scale_r,
        self.specular.g * scale_g,
        self.specular.b * scale_b
    )
    self.diffuse = new_diffuse
    self.specular = new_specular

# =====
# Геометрические объекты
# =====

class IntersectionInfo:
    """Информация о пересечении луча с поверхностью."""
    def __init__(
        self,
        t: float,
        point: Vector3,
        normal: Vector3,
        material: Material,
        emission: Spectrum,
        uv: Tuple[float, float]
    ):

```

```

):
    self.t = t
    self.point = point
    self.normal = normal
    self.material = material
    self.emission = emission
    self.uv = uv

@property
def is_light_source(self) -> bool:
    """Проверка, является ли поверхность источником света"""
    return self.emission.r > 0 or self.emission.g > 0 or
self.emission.b > 0

class Triangle:
    """Треугольный примитив с интерполяцией нормалей"""
    def __init__(
        self,
        v0: Vector3,
        v1: Vector3,
        v2: Vector3,
        n0: Vector3,
        n1: Vector3,
        n2: Vector3,
        material: Material,
        emission: Spectrum = None
    ):
        self.vertices = (v0, v1, v2)
        self.normals = (n0, n1, n2)
        self.material = material
        self.emission = emission if emission else Spectrum.black()
        # Площадь треугольника (для выборки источников света)
        edge1 = v1 - v0
        edge2 = v2 - v0
        self.area = edge1.cross(edge2).length / 2.0

# =====
# Камера и проекция
# =====

class Camera:
    """Точечная камера с перспективной проекцией"""
    def __init__(
        self,
        position: Vector3,
        target: Vector3,
        up: Vector3,
        fov_degrees: float,
        width: int,
        height: int
    ):
        self.position = position
        self.forward = (target - position).normalize()
        self.right = self.forward.cross(up).normalize()
        self.up = self.right.cross(self.forward).normalize()
        self.width = width
        self.height = height
        self.aspect_ratio = width / height

```

```

        # Вычисление размеров пикселя
        fov_rad = math.radians(fov_degrees)
        half_height = math.tan(fov_rad / 2.0)
        half_width = self.aspect_ratio * half_height
        self.pixel_width = half_width * 2 / width
        self.pixel_height = half_height * 2 / height
        # Векторы для плоскости изображения
        self.screen_center = self.position + self.forward
        self.screen_u = self.right * half_width
        self.screen_v = self.up * half_height

    def generate_ray(
        self,
        x: int,
        y: int,
        offset_x: float = 0.0,
        offset_y: float = 0.0
    ) -> Ray:
        """Генерация луча через заданный пиксель"""
        # Координаты с учетом антиалиасинга
        pixel_x = x + offset_x
        pixel_y = y + offset_y
        # Вычисление точки на плоскости изображения
        u = (pixel_x / self.width - 0.5) * 2.0
        v = (0.5 - pixel_y / self.height) * 2.0
        point_on_screen = (
            self.screen_center +
            self.screen_u * u +
            self.screen_v * v
        )
        direction = (point_on_screen - self.position).normalize()
        return Ray(self.position, direction)

# =====
# Сцена и источники света
# =====
class Scene:
    """Трехмерная сцена с использованием trimesh для ускорения"""
    def __init__(self):
        self.triangles = []
        self.lights = []
        self.trimesh_cache = None # Кэшированная сцена для ускорения

    def add_triangle(self, triangle: Triangle):
        """Добавление треугольника в сцену"""
        self.triangles.append(triangle)
        if triangle.emission.r > 0 or triangle.emission.g > 0 or
triangle.emission.b > 0:
            self.lights.append(triangle)
        # Сбрасываем кэш при добавлении нового треугольника
        self.trimesh_cache = None

    def add_sphere(
        self,
        center: Vector3,
        radius: float,
        material: Material,

```

```

        subdivisions=1
    ):
        """Добавление сфер (аппроксимированных треугольниками)"""
        # Вершины икосаэдра
        t = (1.0 + math.sqrt(5.0)) / 2.0
        base_vertices = [
            Vector3(-1, t, 0), Vector3(1, t, 0), Vector3(-1, -t, 0),
            Vector3(1, -t, 0),
            Vector3(0, -1, t), Vector3(0, 1, t), Vector3(0, -1, -t),
            Vector3(0, 1, -t),
            Vector3(t, 0, -1), Vector3(t, 0, 1), Vector3(-t, 0, -1),
            Vector3(-t, 0, 1)
        ]
        base_vertices = [v.normalize() for v in base_vertices]
        faces = [
            (0, 11, 5), (0, 5, 1), (0, 1, 7), (0, 7, 10), (0, 10, 11),
            (1, 5, 9), (5, 11, 4), (11, 10, 2), (10, 7, 6), (7, 1, 8),
            (3, 9, 4), (3, 4, 2), (3, 2, 6), (3, 6, 8), (3, 8, 9),
            (4, 9, 5), (2, 4, 11), (6, 2, 10), (8, 6, 7), (9, 8, 1)
        ]
        for face in faces:
            self._subdivide(
                base_vertices[face[0]],
                base_vertices[face[1]],
                base_vertices[face[2]],
                center,
                radius,
                material,
                subdivisions
            )

    def _subdivide(
        self,
        v1: Vector3,
        v2: Vector3,
        v3: Vector3,
        center: Vector3,
        radius: float,
        material: Material,
        level: int
    ) -> None:
        """Рекурсивное подразделение треугольников для аппроксимации сферы"""
        if level == 0:
            p0 = center + v1 * radius
            p1 = center + v2 * radius
            p2 = center + v3 * radius
            n0 = v1.normalize()
            n1 = v2.normalize()
            n2 = v3.normalize()
            self.add_triangle(Triangle(p0, p1, p2, n0, n1, n2, material))
        else:
            v12 = (v1 + v2).normalize()
            v23 = (v2 + v3).normalize()
            v31 = (v3 + v1).normalize()
            self._subdivide(v1, v12, v31, center, radius, material, level -
1)
            self._subdivide(v2, v23, v12, center, radius, material, level -

```

```

1)         self._subdivide(v3, v31, v23, center, radius, material, level -
1)         self._subdivide(v12, v23, v31, center, radius, material, level -
1)

def _build_trimesh_scene(self):
    """Построение сцены trimesh (один раз при первом вызове)"""
    if self.trimesh_cache is not None:
        return self.trimesh_cache
    vertices = []
    faces = []
    face_to_triangle = []
    for i, tri in enumerate(self.triangles):
        v0, v1, v2 = tri.vertices
        start_idx = len(vertices)
        vertices.append([v0.x, v0.y, v0.z])
        vertices.append([v1.x, v1.y, v1.z])
        vertices.append([v2.x, v2.y, v2.z])
        faces.append([start_idx, start_idx + 1, start_idx + 2])
        face_to_triangle.append(i)
    # Создаем меш trimesh
    mesh = trimesh.Trimesh(vertices=vertices, faces=faces, process=False)
    # Сохраняем структуру с сопоставлениями
    self.trimesh_cache = {
        'mesh': mesh,
        'face_to_triangle': face_to_triangle,
        'vertices': vertices,
        'faces': faces
    }
    return self.trimesh_cache

def intersect(self, ray: Ray) -> Optional[IntersectionInfo]:
    """Оптимизированный поиск пересечений с использованием trimesh"""
    # Используем trimesh для быстрого поиска пересечений
    scene_data = self._build_trimesh_scene()
    mesh = scene_data['mesh']
    face_to_triangle = scene_data['face_to_triangle']
    # Преобразуем луч для trimesh
    ray_origin = np.array([ray.origin.x, ray.origin.y, ray.origin.z]),
dtype=np.float64)
    ray_direction = np.array([ray.direction.x, ray.direction.y,
ray.direction.z]), dtype=np.float64)
    # Используем оптимизированный поиск trimesh
    locations, index_ray, index_tri = mesh.ray.intersects_location(
        ray_origins=ray_origin,
        ray_directions=ray_direction,
        multiple_hits=False
    )
    if len(locations) == 0:
        return None
    # Нашли пересечение - берем первый (ближайший)
    triangle_idx = face_to_triangle[index_tri[0]]
    triangle = self.triangles[triangle_idx]
    point = Vector3(locations[0][0], locations[0][1], locations[0][2])
    # Вычисляем барицентрические координаты
    A, B, C = triangle.vertices

```

```

u, v, w = self.compute_barycentric_coordinates(point, A, B, C)
# Интерполируем нормаль
normal = (
    triangle.normals[0] * u +
    triangle.normals[1] * v +
    triangle.normals[2] * w
)
norm_length = normal.length
if norm_length < 1e-8:
    # Вырожденная нормаль, используем нормаль из треугольника
    normal = triangle.normals[0]
else:
    normal = normal / norm_length
# Проверяем, что луч приходит с лицевой стороны треугольника
if ray.direction.dot(normal) >= 0:
    # Луч приходит с тыльной стороны, игнорируем пересечение
    return None
# Вычисляем расстояние t
t = (point - ray.origin).length
# Возвращаем информацию о пересечении
return IntersectionInfo(
    t=t,
    point=point,
    normal=normal,
    material=triangle.material,
    emission=triangle.emission,
    uv=(v, w) # координаты для вершин B и C
)

@staticmethod
def compute_barycentric_coordinates(P: Vector3, A: Vector3, B: Vector3,
C: Vector3) -> Tuple[float, float, float]:
    """Вычисление барицентрических координат точки P относительно
треугольника ABC"""
    # Решение системы уравнений через проекции
    v0 = B - A
    v1 = C - A
    v2 = P - A
    # Вычисление через скалярные произведения
    d00 = v0.dot(v0)
    d01 = v0.dot(v1)
    d11 = v1.dot(v1)
    d20 = v2.dot(v0)
    d21 = v2.dot(v1)
    denom = d00 * d11 - d01 * d01
    if abs(denom) < 1e-8:
        return 1.0, 0.0, 0.0
    # Вычисление координат
    v = (d11 * d20 - d01 * d21) / denom
    w = (d00 * d21 - d01 * d20) / denom
    u = 1.0 - v - w
    # Обрезаем до [0,1] для устойчивости
    u = max(0.0, min(1.0, u))
    v = max(0.0, min(1.0, v))
    w = max(0.0, min(1.0, w))
    # Нормализуем, чтобы сумма была равна 1
    total = u + v + w

```

```

        if total > 0:
            u /= total
            v /= total
            w /= total
        return u, v, w

# =====
# Трассировщик путей
# =====
class PathTracer:
    """Трассировщик путей"""
    def __init__(
        self,
        scene: Scene,
        camera: Camera,
        width: int,
        height: int,
        samples_per_pixel: int = 16,
        max_depth: int = 5
    ):
        self.scene = scene
        self.camera = camera
        self.width = width
        self.height = height
        self.samples = samples_per_pixel
        self.max_depth = max_depth
        # Инициализация изображения
        self.image = np.zeros((height, width, 3), dtype=np.float32)

    def trace_path(self, ray: Ray, depth: int = 0) -> Spectrum:
        """Основной алгоритм трассировки пути"""
        if depth >= self.max_depth:
            return Spectrum.black()
        hit = self.scene.intersect(ray)
        if not hit:
            return Spectrum.black()
        # Если попали на источник света - возвращаем его излучение
        if hit.is_light_source:
            return hit.emission
        material = hit.material
        point = hit.point
        normal = hit.normal
        # Русская рулетка
        if depth > 3: # Начинаем применять после 3-го отскока
            survival_prob = 0.8
            if random.random() > survival_prob:
                return Spectrum.black()
        else:
            survival_prob = 1.0
        # Выбор типа отражения (диффузное или зеркальное)
        diffuse_weight = material.diffuse.luminance
        specular_weight = material.specular.luminance
        total_weight = diffuse_weight + specular_weight
        if total_weight < 1e-6:
            return Spectrum.black()
        # Нормализуем вероятности
        diffuse_prob = diffuse_weight / total_weight

```



```

        specular_prob = specular_weight / total_weight
        # Выбор случайного направления
        if random.random() < diffuse_prob:
            # Диффузное отражение (Lambert) - выбрано с вероятностью
diffuse_prob
            new_direction = Vector3.random_cosine_direction(normal)
            pdf = max(1e-8, new_direction.dot(normal) / math.pi)
            if pdf < 1e-12:
                return Spectrum.black()
            new_ray = Ray(point + normal * 1e-4, new_direction)
            cosine = abs(new_direction.dot(normal))
            incoming = self.trace_path(new_ray, depth + 1)
            brdf = material.diffuse / math.pi
            # Косвенное освещение (importance sampling по BRDF)
            # Делим на вероятность выбора диффузного отражения
            indirect_light = incoming * brdf * cosine / (pdf * diffuse_prob)
            # Прямое освещение (next event estimation)
            # Также делим на вероятность выбора диффузного отражения
            direct_light = self.sample_direct_lighting(point, normal) * brdf
/ diffuse_prob
            result = indirect_light + direct_light
        else:
            # Зеркальное отражение - выбрано с вероятностью specular_prob
            reflected = ray.direction.reflect(normal).normalize()
            new_ray = Ray(point + normal * 1e-4, reflected)
            incoming = self.trace_path(new_ray, depth + 1)
            # Для идеального зеркала BRDF = specular * delta-функция
            # PDF для зеркального отражения = 1 (детерминированное
направление)
            # Делим на вероятность выбора зеркального отражения
            result = incoming * material.specular / specular_prob
            # Масштабируем по вероятности выживания (русская рулетка)
            if depth > 3:
                result = result / survival_prob
            return result

    def sample_direct_lighting(self, point: Vector3, normal: Vector3) ->
Spectrum:
        """Выборка прямого освещения (один сэмпл)"""
        if not self.scene.lights:
            return Spectrum.black()
        # Выбираем случайный источник света
        light = random.choice(self.scene.lights)
        # Выбор случайной точки на источнике света
        u = random.random()
        v = random.random()
        if u + v > 1:
            u = 1 - u
            v = 1 - v
        w = 1 - u - v
        # Вычисляем точку на источнике света
        light_point = (
            light.vertices[0] * w +
            light.vertices[1] * u +
            light.vertices[2] * v
        )
        to_light = light_point - point

```

```

distance = to_light.length
if distance <= 1e-8:
    return Spectrum.black()
light_dir = to_light / distance
cos_surface = abs(normal.dot(light_dir))
if cos_surface <= 0:
    return Spectrum.black()
# Проверка видимости
eps = 1e-4
origin = point + normal * eps
shadow_ray = Ray(origin, light_dir)
shadow_hit = self.scene.intersect(shadow_ray)
if shadow_hit and shadow_hit.t < distance - 1e-4:
    return Spectrum.black()
# Вычисление геометрического фактора
light_normal = (
    light.normals[0] * w +
    light.normals[1] * u +
    light.normals[2] * v
).normalize()
cos_light = abs(light_normal.dot(-light_dir))
if cos_light <= 0:
    return Spectrum.black()
edge1 = light.vertices[1] - light.vertices[0]
edge2 = light.vertices[2] - light.vertices[0]
light_area = edge1.cross(edge2).length / 2.0
if light_area <= 0:
    return Spectrum.black()
# PDF для выборки света
light_pdf = 1.0 / (len(self.scene.lights) * light_area)
# Формула освещения
geometry_factor = cos_surface * cos_light / (distance * distance)
if geometry_factor <= 0:
    return Spectrum.black()
# Возвращаем вклад сэмпла
return light.emission * geometry_factor / light_pdf

def render(self) -> np.ndarray:
    """Основной метод рендеринга"""
    print(f"Начало рендеринга: {self.width}x{self.height}")
    print(f"Сэмплов на пиксель: {self.samples}")
    print(f"Глубина трассировки: {self.max_depth}")
    print(f"Треугольников в сцене: {len(self.scene.triangles)}")
    start_time = time.time()
    total_pixels = self.width * self.height
    pixel_counter = 0
    for y in range(self.height):
        for x in range(self.width):
            color = Spectrum.black()
            # Мультисемплирование на уровне пикселя (антиалиасинг)
            for _ in range(self.samples):
                # Антиалиасинг
                offset_x = random.random() - 0.5
                offset_y = random.random() - 0.5
                ray = self.camera.generate_ray(x, y, offset_x, offset_y)
                path_color = self.trace_path(ray)
                color += path_color

```

```

        # Усреднение по количеству сэмплов
        color = color / self.samples
        self.image[y, x] = [color.r, color.g, color.b]
        pixel_counter += 1
        if pixel_counter % 400 == 0:
            progress = pixel_counter / total_pixels * 100
            elapsed = time.time() - start_time
            print(
                f"\rПрогресс: {progress:.1f}%
({pixel_counter}/{total_pixels}), время: {elapsed:.1f}с",
                end="", flush=True
            )
            elapsed = time.time() - start_time
            print(f"\nРендеринг завершен за {elapsed:.2f} секунд")
            return self.image

# =====
# Утилиты обработки изображений
# =====
def tonemap(
    image_data: np.ndarray,
    exposure: float = 1.0,
    gamma: float = 2.2
) -> np.ndarray:
    """ACES тонмаппинг"""
    image_data = image_data * exposure
    # ACES approximation
    A = 2.51
    B = 0.03
    C = 2.43
    D = 0.59
    E = 0.14
    image_data = (image_data * (A * image_data + B)) / (image_data * (C *
image_data + D) + E)
    # Отсечение и гамма
    image_data = np.clip(image_data, 0.0, 1.0)
    image_data = np.power(image_data, 1.0 / gamma)
    return image_data

def save_ppm(filename: str, image_data: np.ndarray):
    """Сохранение изображения в формате PPM"""
    height, width, _ = image_data.shape
    # Конвертация в 8-битный формат
    image_8bit = (np.clip(image_data, 0, 1) * 255).astype(np.uint8)
    with open(filename, 'wb') as f:
        # Заголовок PPM
        header = f"P6\n{width} {height}\n255\n"
        f.write(header.encode('ascii'))
        # Данные пикселей
        for y in range(height):
            for x in range(width):
                r, g, b = image_8bit[y, x]
                f.write(bytes([r, g, b]))
    print(f"Изображение сохранено: {filename}")

# =====
# Построение конкретной сцены

```

```

# =====
class CornellBoxScene:
    def __init__(self):
        self.red_material = Material(
            diffuse=Spectrum(0.65, 0.05, 0.05),
            specular=Spectrum(0.0, 0.0, 0.0),
            name="Red Material"
        )
        self.green_material = Material(
            diffuse=Spectrum(0.12, 0.45, 0.15),
            specular=Spectrum(0.0, 0.0, 0.0),
            name="Green Material"
        )
        self.white_material = Material(
            diffuse=Spectrum(0.9, 0.9, 0.9),
            specular=Spectrum(0.0, 0.0, 0.0),
            name="White Material"
        )
        self.mirror_white_material = Material(
            diffuse=Spectrum(0.9, 0.9, 0.9),
            specular=Spectrum(0.1, 0.1, 0.1),
            name="Mirror White Material"
        )
        self.mirror_material = Material(
            diffuse=Spectrum(0.01, 0.01, 0.01),
            specular=Spectrum(0.99, 0.99, 0.99),
            name="Mirror Material"
        )
        self.light = Material(
            diffuse=Spectrum(0.0, 0.0, 0.0),
            specular=Spectrum(0.0, 0.0, 0.0),
            emission=Spectrum(6.0, 6.0, 6.0),
            name="Light"
        )
        self.scene = self._create_cornell_box()

    def _create_cornell_box(self) -> Scene:
        """Создание сцены Корнелл Бокс"""
        scene = Scene()
        # Векторы нормалей
        up = Vector3(0, 1, 0)
        down = Vector3(0, -1, 0)
        left = Vector3(1, 0, 0)
        right = Vector3(-1, 0, 0)
        back = Vector3(0, 0, -1)
        # Пол (белый) - нормали вверх
        scene.add_triangle(Triangle(
            Vector3(-1, -1, -1), Vector3(1, -1, -1), Vector3(-1, -1, 1),
            up, up, up, self.white_material
        ))
        scene.add_triangle(Triangle(
            Vector3(1, -1, -1), Vector3(1, -1, 1), Vector3(-1, -1, 1),
            up, up, up, self.white_material
        ))
        # Потолок (белый) - нормали вниз
        scene.add_triangle(Triangle(
            Vector3(-1, 1, -1), Vector3(-1, 1, 1), Vector3(1, 1, -1),

```

```

        down, down, down, self.white_material
    ))
    scene.add_triangle(Triangle(
        Vector3(1, 1, -1), Vector3(-1, 1, 1), Vector3(1, 1, 1),
        down, down, down, self.white_material
    ))
    # Левая стена (красная) - нормаль вправо
    scene.add_triangle(Triangle(
        Vector3(-1, -1, -1), Vector3(-1, -1, 1), Vector3(-1, 1, -1),
        left, left, left, self.red_material
    ))
    scene.add_triangle(Triangle(
        Vector3(-1, 1, -1), Vector3(-1, -1, 1), Vector3(-1, 1, 1),
        left, left, left, self.red_material
    ))
    # Правая стена (зеленая) - нормаль влево
    scene.add_triangle(Triangle(
        Vector3(1, -1, -1), Vector3(1, 1, -1), Vector3(1, -1, 1),
        right, right, right, self.green_material
    ))
    scene.add_triangle(Triangle(
        Vector3(1, 1, -1), Vector3(1, 1, 1), Vector3(1, -1, 1),
        right, right, right, self.green_material
    ))
    # Задняя стена (белая-слегка зеркальная) - нормаль вперед
    scene.add_triangle(Triangle(
        Vector3(-1, -1, 1), Vector3(1, -1, 1), Vector3(-1, 1, 1),
        back, back, back, self.mirror_white_material
    ))
    scene.add_triangle(Triangle(
        Vector3(1, -1, 1), Vector3(1, 1, 1), Vector3(-1, 1, 1),
        back, back, back, self.mirror_white_material
    ))
    # Источник света (на потолке)
    light_size = 0.5
    scene.add_triangle(Triangle(
        Vector3(-light_size, 0.999, -light_size),
        Vector3(-light_size, 0.999, light_size),
        Vector3(light_size, 0.999, -light_size),
        down, down, down, self.light, self.light.emission
    ))
    scene.add_triangle(Triangle(
        Vector3(light_size, 0.999, -light_size),
        Vector3(-light_size, 0.999, light_size),
        Vector3(light_size, 0.999, light_size),
        down, down, down, self.light, self.light.emission
    ))
    # добавление объектов на сцену
    scene.add_sphere(
        Vector3(0.4, -0.7, -0.2),
        0.3,
        self.white_material,
        1
    )
    scene.add_sphere(
        Vector3(-0.4, 0.0, 0.2),
        0.2,

```

```

        self.mirror_material,
        1
    )
    return scene

# =====
# Интерфейс командной строки
# =====
def main():
    """Основная функция программы."""
    parser = argparse.ArgumentParser(
        description="Path Tracer with Global Illumination",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Примеры использования:
# Быстрый тест
python path_tracer.py --width 200 --height 200 --samples 4 --max-depth 3

# Среднее качество
python path_tracer.py --width 500 --height 500 --samples 16 --max-depth 5

# Высокое качество
python path_tracer.py --width 600 --height 600 --samples 24 --max-depth 8
        """
    )
    parser.add_argument(
        "--width", type=int, default=500, help="Ширина изображения"
    )
    parser.add_argument(
        "--height", type=int, default=500, help="Высота изображения"
    )
    parser.add_argument(
        "--samples", type=int, default=16, help="Количество сэмплов на пиксель"
    )
    parser.add_argument(
        "--max-depth", type=int, default=8, help="Максимальная глубина трассировки"
    )
    parser.add_argument(
        "--fov", type=float, default=45.0, help="Поле зрения камеры"
    )
    parser.add_argument(
        "--exposure", type=float, default=1.0, help="Экспозиция для тонмэппинга"
    )
    parser.add_argument(
        "--gamma", type=float, default=2.2, help="Гамма-коррекция"
    )
    parser.add_argument(
        "--output", type=str,
        default=f"render-{datetime.now().strftime('%Y-%m-%d_%H-%M-%S')}.ppm",
        help="Имя выходного файла"
    )
    args = parser.parse_args()
    print(f"Разрешение:      {args.width}x{args.height}")
    print(f"Сэмплов на пиксель:  {args.samples}")

```

```

print(f"Глубина трассировки: {args.max_depth}")
print(f"Поле зрения камеры: {args.fov}")
print(f"Экспозиция: {args.exposure}")
print(f"Гамма-коррекция: {args.gamma}")
# Создание сцены
print("\nСоздание сцены...")
cornell_box_scene = CornellBoxScene()
scene = cornell_box_scene.scene
print(f"Треугольников: {len(scene.triangles)}")
print(f"Источников света: {len(scene.lights)}")
# Создание камеры и трассировщика
camera_pos = Vector3(0, 0, -3)
camera_target = Vector3(0, 0, 0)
camera_up = Vector3(0, 1, 0)
camera = Camera(
    camera_pos,
    camera_target,
    camera_up,
    args.fov,
    args.width,
    args.height
)
tracer = PathTracer(
    scene,
    camera,
    args.width,
    args.height,
    args.samples,
    args.max_depth
)
# Рендеринг
print("\nНачало рендеринга...")
image_data = tracer.render()
# Тонмэппинг
print("\nПрименение тонмэппинга...")
tone_mapped = tonemap(image_data, args.exposure, args.gamma)
# Сохранение результата
save_ppm(args.output, tone_mapped)
print(f"\nРендеринг завершен!")

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\nРендеринг прерван пользователем")
        sys.exit(0)
    except Exception as e:
        print(f"\nОшибка: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)

```

Вывод

Разработанная программа успешно реализует алгоритм трассировки путей для создания фотореалистичных изображений с учетом глобального освещения. Программа демонстрирует:

1. Корректную физическую модель освещения и материалов
2. Эффективную реализацию с использованием trimesh для ускорения вычислений
3. Современные методы выборки для уменьшения шума
4. Гибкую систему настройки параметров рендеринга через командную строку
5. Поддержку тонмаппинга и гамма-коррекции для корректного отображения

Изображение Cornell Box, полученное в результате рендеринга, демонстрирует все ключевые аспекты глобального освещения, подтверждая корректность реализации алгоритма трассировки путей.